Adversarial training defense techniques retrain the attack target model by using both benign training set and adversarial examples generated using known attack algorithms [1], [18] in order to improve the generalization of the target model in the presence of adversarial attacks. PGD adversarial training with random restart [10] and multi-model-training on adversarial examples [19] are the two most recent and more advanced adversarial training algorithms. Adversarial training can be seen as giving the target model a set of flu-shots, one per attack algorithm, and thus, adversarial training defense can be effective against known attacks. However, it also suffers from the attack-dependent training and fails to generalize and against other attacks.

Gradient masking defense techniques add an additional layer of training to reduce the sensitivity of a trained model in response to small changes in input data. The main idea is to obfuscate the gradient information or use a near-zero gradient to off-set or minimize the impact of gradient information manipulation performed by an adversary. [20] adds a gradient penalty term in the model training objective, which is a summation of the layer-wise Frobenius norm of the Jacobian matrix. The gradient penalty makes the target model less sensitive to small perturbations in input, but notably reduces the accuracy and learning effectiveness of the target model. Defensive distillation [21] replaces the last layer with a defensive softmax function and a temperature knot to control the extent of distillation after training.

Input transformation applying careful input manipulation techniques to the input data before sending it to the target prediction model. Some popular image processing techniques like binary filters and median smoothing filters are employed in [22] to perform feature squeezing on input data. Other input transformation solutions include PixelDefend [23], information encoding [24] image quilting and total variation minimization [25], randomness [26] or region sampling [27]. The dimension reduction techniques include the PCA projection of benign training data with the top k principle axes [28] and the DNN denoising autoencoders [20], [29], [30], which learns the latent space representation on training data and leverages the latent patterns to detect adversarial inputs.

Defense on OOD Inputs. The first out-of-distribution detection method, proposed by the work [14], adds small perturbations to the input data by softmax-controlled input preprocessing and is often referred to as the baseline. One way to improve the baseline method is to combine the input noise injection with output temperature scaling by distillation temperature control [6]. However, both detection methods rely on a proper setting of the input noise amount parameter and the output temperature parameter, which are dataset-specific. Another proposal utilizes the Mahalanobis distance with respect to the closest class conditional distribution to flag both out-of-distribution examples and adversarial perturbed examples [5]. Other methods include training robust classifier against out-of-distribution data [31], GAN-based detection [32] and semantic representation [33].

Limitations of Existing Defenses. (1) Existing defense methods are sensitive to certain magic parameters inherent in their design, such as the percentage of adversarial examples in a batch in adversarial training, the temperature in Defensive Distillation, the detection threshold trained on benign dataset and adversarial inputs in both input transformation and denoising auto-encoder detector defense. Such dataset-specific and/or attack algorithm-specific control parameters make the defense methods non-adaptive [34]. (2) Existing detection only defenses are either limited to either out-of-distribution inputs or adversarial examples, whereas other defense methods can only defend adversarial examples but not able to detect OOD input. We argue that a defense system is practical if it is attack-independent and can generalize over attack algorithms. A robust defense approach should not depend on finding the attack-specific or dataset-specific control knob (threshold) to distinguish the adversarial inputs from the benign inputs and to detect and flag those out of distribution inputs. XEnsemble is developed to promote high robustness and high defensibility of the DNN model against both adversarial examples and out-of-distribution inputs.

3 XEnsemble Solution Approach

3.1 Overview

During the prediction phase, adversarial examples are sent to the prediction APIs to cause misclassification. Our past work [4] identified that different DNN models tend to have different gradient information and thus an adversarial example is often misclassified inconsistently across the different models. For untargeted attacks, such adversarial divergence is reflected by different attack destination classes of the same input. For targeted attacks, the same adversarial example often results in different most-likely and least-likely attack target classes under different prediction models. Although adversarial examples are transferable from one model to another [7], the transferability across different DNN models is not consistent, and the transferability is relatively more severe for untargeted attacks but less so for targeted attacks. For the same adversarial example, the probability of being misclassified by different models to the same wrong class label is not high. Meanwhile, different inputs from the same class also demonstrate some inconsistency under a single model: different destination labels in untargeted attacks and different most-likely and least-likely attack labels in targeted attacks [4]. For OOD inputs, we observe that the provided wrongly-predicted label, even under noise with the same distribution and different appearance, is different due to the exhibited uncertainty [35]. Since both adversarial example and OOD input cause misclassification and both have a low probability of being misclassified to the same wrong class, such prediction discrepancy can be utilized to build a robust prediction model to defend the out-of-distribution input.

Motivated by the characterization of adversarial examples [4], we argue that a robust defense should meet the following two design objectives simultaneously: (1) It should provide a uniform defense architecture that can generalize over both attack algorithms and datasets, and be capable of defending against both adversarial examples and out-of-distribution examples simultaneously. (2) It should be capable of distinguishing the adversarial examples that can be repaired from those that cannot, aiming to maximize the auto-repairing effectiveness. It should also be capable of maximizing the detection rate on the out-of-distribution examples to reduce or minimize the misclassification rate.

We propose to develop XEnsemble, an input-output model verification ensemble defense methodology. XEnsemble is by design unique in two aspects. First, it promotes the composition of disagreement diversity ensembles using high accuracy base models instead of weak models. Second, it combines input verification with output verification using ensemble techniques based on both denoising and failure independent redundancy. XEnsemble can protect a well trained DNN model at the prediction phase without re-training the model using adversarial examples. XEnsemble can be easily added as a plug-in to existing modern

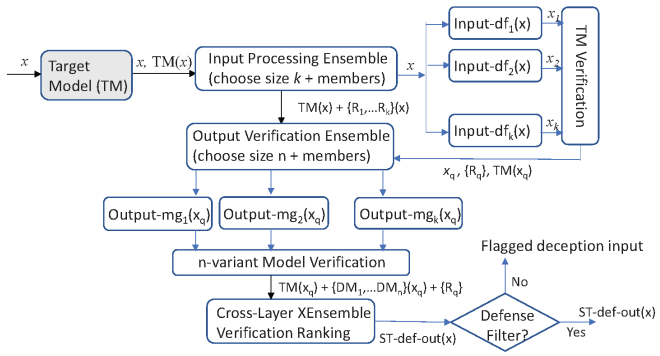

Fig. 2: Architecture of the XEnsemble defense system

neural network architecture. Figure 2 shows an architectural sketch of the XEnsemble system. It combines diverse input denoising techniques with disagreement diversity output verification techniques. An input x submitted to a DNN model under protection (target model) is first processed by k diverse input denoising techniques, denoted by $input_df_1, input_df_2, \ldots, input_df_k$ ($k \geq 3$). We require to choose only those techniques such that both the target model and the ensemble member models have high test accuracy, on par with their accuracy performance on the original dataset. Second, for each of the k denoising input versions of x, say x_q, it will be sent to the output-verification ensemble layer, which will select n model verifiers, denote by $output_mg_1, output_mg_2, \ldots, output_mg_n$ ($n \geq 3$) for output model verification. XEnsemble will verify, repair and output the defense-approved prediction outcome of the target model, denoted by ST-def-out(x). The two ensemble size parameters k and n for input denoising process and output model verification are chosen either randomly or by using specific diversity ranking criteria [36]. Consider output model verification, we first constructed a pool of base models that have diverse network structure or backbone algorithm compared to the target model. This base model pool can be obtained by training new models from scratch or by leveraging the model zoo of pre-trained models for the same task [22]. Then, we compute the ensemble diversity score for each combination of ensemble teams of size 2 or larger from the base model pool. For a base model pool of size M=3, 5 or 10, we will have a total of 3, 26, 1013 ensemble teams. In our first prototype of XEnsemble, we use the κ method [37] to compute the κ value for each team as the prediction disagreement measure. Third, upon receiving multiple versions of the input x from the input denoising layer, we select an ensemble team from the list of top-ranked ensembles by their ensemble diversity scores, we send each of the k input versions to the chosen diversity ensemble and collect the total of $k \times n$ prediction results. We combine these results by ranking them against a pre-defined confidence level and output the verified prediction results if the degree of the prediction agreement is higher than the pre-defined confidence level, and otherwise, we flag the input (when the agreement is below the threshold). Algorithm 1 provides a procedural sketch of the XEnsemble method.

We would like to note that the concept of input denoising and the ensemble diversity are general and applicable to different data modalities, such as image, audio, and text. For example, quantization, filtering, and Gaussian smoothing can be applied to image data; spectral subtraction and Wiener filtering are effective

Algorithm 1: Workflow of XEnsemble

Inputs:
x_0: query data, mg_0: target prediction model, k: number of input-denoising modules, n: number of output verification models, T: L_1 ranking confidence level
ensemble preparation
select k input-denoising methods $df_1, df_2, ..., df_k$ with expertise in processing different kinds of noise.
generate a base model pool according to the target model and the prediction task
from the base model pool, create a diversity ensemble model pool using κ metric or other diversity metrics.
given the diversity ensemble model pool, get top m model ensembles with n models: $\{mg_1, mg_2, .., .mg_n\}_{1,...,m}$.
ensemble prediction
generate multiple denoised version of querry: $x_1, ..., x_k$ from x_0 via different noise reduction techniques $df_1, df_2, ..., df_k$.
feed $x_0, x_1, ..., x_k$ to all models in the selected ensembles $\{mg_1, mg_2, .., .mg_n\}_{1,...,m}$ and get ensemble prediction results
ranking prediction made by $\{mg_1(x_0), .., .mg_n(x_0)\}_i, ..., \{mg_1(x_k), .., .mg_n(x_k)\}_i$, where $i \in \{1, ..., m\}$
if L_1 distance ranking of the selected ensemble $<$ ranking confidence level **then**
    output XEnsemble verified prediction results
else
    flag the deception input and output a watchout alert.
end if

for audio data. In the rest of the paper, we describe in detail our prototype implementation system using the image classification learning as an example application domain. Hence, the input denoising ensemble methods and the output verification ensemble methods are tailored accordingly. We consider three threat scenarios for XEnsemble system when an adversary attacks victim model (target model) based on the level of knowledge about the XEnsemble defense system: (1) zero knowledge (black-box threat model), (2) partial knowledge (grey-box threat model), and (3) full knowledge (insider adversary) (white box threat model). We first show that when attackers can launch white box attack to the target model but have no prior knowledge of XEnsemble protection. This is the common scenario that is used in the literature for defense proposals. We will also discuss the scenario where attackers have partial (grey box threat) or full knowledge (white box threat) of XEnsemble defense in addition to the target victim model.

3.2 Input Verification Ensemble

The main goal of input verification is to apply multiple data modality-specific input noise reduction techniques to clean the input and produce denoised versions of the input, aiming to remove adversarial perturbations or detect the out of distribution inputs and to make the first attempt to prevent adversarial misclassification. While the benign input and their denoised version are prone to be correctly classified, Our work [4] shows that the adversarial example and its denoised counterparts will demonstrate prediction discrepancy. While the location of the attack perturbation is carefully chosen and the amount is minimized to meet the imperceptibility, input denoising is applied uniformly on the entire image and is by design to preserve the semantic and visualization quality of the input image. We argue that the input noise reduction techniques should be chosen to preserve certain verifiable properties, such as the test accuracy of the target model on benign inputs, while capable of removing the adverse effect from an adversarial input.

	Attack	benign	FGSM	BIM	PGD	CW_∞		CW_2		CW_0		JSMA		average
			UA			most	LL	most	LL	most	LL	most	LL	
CIFAR-10	no defense	**0.9484**	0.15	0.08	0	0	0	0	0	0	0	0	0	0.021
	quan-1-bit	0.2111	0.18	0.17	0.14	0.15	0.17	0.17	0.2	0.16	0.14	0.15	0.16	0.163
	quan-4-bit	0.9311	0.21	0.23	0.17	0.43	0.73	0.54	0.82	0.08	0.08	0.3	0.2	0.345
	medFilter-2*2	0.8929	0.38	0.56	0.40	**0.73**	0.84	**0.79**	0.84	**0.84**	**0.86**	**0.89**	0.74	**0.715**
	medFilter-3*3	0.7502	**0.42**	**0.63**	0.66	0.69	0.69	0.72	0.73	0.77	0.77	0.75	0.75	0.626
	NLM-11-3-2	0.9421	0.2	0.25	**0.67**	0.36	0.74	0.38	0.78	0	0.01	0.22	0.12	0.339
	NLM-11-3-4	0.9118	0.27	0.46	0.35	0.57	0.85	0.64	0.9	0.05	0.11	0.46	0.32	0.453
	NLM-13-3-2	0.9414	0.19	0.26	0.54	0.37	0.75	0.38	0.81	0	0.01	0.27	0.12	0.336
	NLM-13-3-4	0.9083	0.27	0.45	0.35	0.59	0.85	0.63	0.87	0.05	0.13	0.47	0.32	0.453
	rotation_-12	0.8335	0.33	0.52	0.54	0.61	0.73	0.65	0.76	0.32	0.5	0.47	0.34	0.525
	rotation_-9	0.8529	0.28	0.5	0.57	0.64	0.79	0.67	0.8	0.27	0.45	0.49	0.36	0.529
	rotation_3	0.853	0.27	0.33	0.50	0.55	0.71	0.58	0.7	0.15	0.4	0.36	0.22	0.434
	rotation_6	0.8493	0.26	0.41	0.50	0.57	0.7	0.62	0.72	0.22	0.47	0.4	0.3	0.470
med_2*2, rot_-12, NLM-13-3-4		0.8927	0.38	0.60	**0.67**	0.72	**0.89**	0.75	**0.91**	0.36	0.84	0.67	**0.76**	0.686
ImageNet	no defense	0.695	0.01	0	0	0	0.04	0	0.06	0	0	-	-	0.012
	quan-1-bit	0.24	0.28	0.26	0.28	0.27	0.28	0.27	0.29	0.28	0.22	-	-	0.270
	quan-4-bit	0.695	0.05	0.05	0.05	0.31	0.76	0.47	0.82	0.1	0.58	-	-	0.354
	medfilter-2*2	0.65	0.22	0.28	0.26	0.68	0.82	0.74	0.85	0.84	0.85	-	-	0.616
	medfilter-3*3	0.61	0.33	0.41	0.37	0.7	0.8	0.73	0.78	0.79	0.82	-	-	0.637
	NLM-11-3-2	0.7	0.05	0.09	0.08	0.28	0.75	0.43	0.87	0.04	0.25	-	-	0.316
	NLM-11-3-4	0.66	0.1	0.25	0.21	0.59	0.83	0.68	0.84	0.2	0.5	-	-	0.467
	NLM-13-3-2	0.7	0.06	0.09	0.09	0.32	0.75	0.46	0.87	0.04	0.27	-	-	0.328
	NLM-13-3-4	0.665	0.11	0.26	0.23	0.59	0.84	0.68	0.87	0.2	0.51	-	-	0.477
	rotation_-12	0.62	0.41	0.55	0.51	0.74	0.74	0.73	0.78	0.68	0.64	-	-	0.642
	rotation_-9	0.635	0.39	0.53	0.48	0.77	0.79	0.78	0.8	0.69	0.72	-	-	0.661
	rotation_3	0.68	0.29	0.44	0.40	0.71	0.83	0.76	0.82	0.65	0.78	-	-	0.631
	rotation_6	0.68	0.33	0.49	0.46	0.78	0.85	0.79	0.83	0.72	0.74	-	-	0.666
med-3*3, rot_-9, rot_6		**0.75**	**0.53**	**0.64**	**0.65**	**0.90**	**0.92**	**0.89**	**0.93**	**0.87**	**0.89**	-	-	**0.802**

TABLE 2: Defense accuracy of different feature denoising techniques for MNIST, CIFAR-10, and ImageNet

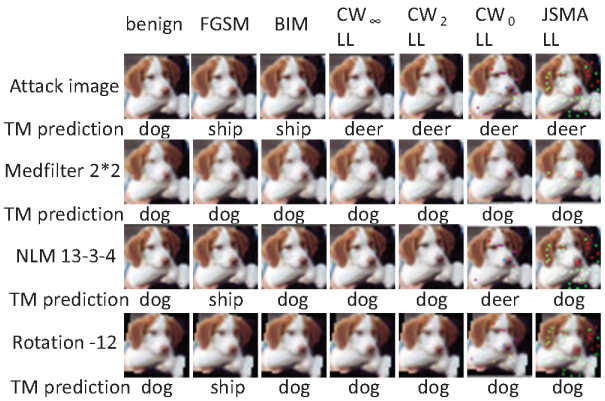

Fig. 3: CIFAR-10 input-layer defense under 6 attacks.

For image classifiers, image smoothing and image augmentation techniques are popular for image noise reduction [38]. The former includes pixel quantization by color bit depth reduction, local spatial smoothing, and non-local spatial smoothing. The latter include image rotation, image cropping, and rescaling, image quilting and compression. Although these techniques are not designed to remove the injected noise directly, they can be utilized to make the perturbation less or no longer effective. In the first implementation of XEnsemble, four techniques are implemented using the Scipy library [39] and OpenCV [40].

Rotation method is a standard image geometric transformation technique provided in SciPy library, rotation preserves the geometric distance of the image and does not change the neighborhood information for most of the pixels except for the corner cases. In our prototype defense, the rotation degree is varied from -12 to 12 with an interval of 3 degree.

The **Color-depth reduction** is another technique that reduces the color depth of 8 bits ($2^8 = 256$ values) to i bits (2^i values). If $i = 1$, then the bit quantization will replace the [0,255] space to 1-bit encoding with 2 values: it takes 0 when the nearby pixel value is smaller than 127 and takes 1 when pixel values are in the range of [128, 255]. We use quan-i-bit to denote the quantization of the input image from the original 8-bit encoded color depth to i-bit ($1 \leq i < 8$).

The **local spatial smoothing** technique uses nearby pixels to smooth each pixel, with Gaussian, mean or median smoothing. A median filter runs a sliding window over each pixel of the image, where the center pixel is replaced by the median value of the neighboring pixels within the window. The size of the window is a configurable parameter, ranging from 1 up to the image size. MedFilter-$k*k$ denotes the median filter with neighborhood kernel size $k * k$. A square shape window size, e.g., 2×2 or 3×3, is often used with reflect padding.

The **Non-local spatial smooth (NLM)** technique smooths over similar pixels by exploring a larger neighborhood (11×11 search window) instead of just nearby pixels and replaces the center patch (size of 3×3) with the (Gaussian) weighted average of those similar patches in the search window. NLM-a-b-c denotes non-local means smoothing filter with searching window size $a*a$, patch size $b * b$ and Gaussian distribution parameter c. NLM 11-3-4 refers to the NLM filter with 11×11 search window, 3×3 patch size and the filter strength of 4.

Table 2 shows the results of input-denoising layer. In this set of experiments, we feed each denoised version to the target model and we show the performance of individual denoising technique and one example denoising ensemble for CIFAR-10 and ImageNet respectively. We make several interesting observations. *First*, employing an input noise reduction technique can always improve the robustness of the target model compared to the no-defense scenario. However, there is no single input denoising method that is effective across all 11 attacks. Each technique is good at removing some types of noise but not the other. *Second*, not all noise reduction techniques exhibit consistent performance across different datasets trained with different models (e.g., CIFAR-10 with DenseNet and ImageNet with MobileNet as shown in Table 3). Another interesting observation is that the least-likely attacks are relatively easier to defend than the most-likely attacks.